# Git Fundamentals

A guide to git basic concepts and how they are used in Einstein

Medtronic

# Course overview

This course assumes you are familiar with basic git operations, such as clone, commit, push and pull

It explains the underlying objects behind these operations and how they behave, particularly in a large software project with a central git server.

It covers aspects that we don't necessarily use in Einstein, but it also includes some Einstein-specific details.  These Einstein-specific details are highlighted.

After introduction git concepts, I will go over three specific Einstein git workflows that are in the Confluence **Einstein Software DevOps** folder:

- Feature branch management
- Staging branch merging
- Cherrypicking fixes to release branches

There is also an Einstein git quick reference card in the same Confluence folder

# Goals

Be able to follow the procedures for making and delivering changes in the Einstein codebase.

Set up and manage local workspaces.

Be able to troubleshoot and recover from git issues, including merge issues

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Repos

The Einstein codebase is split into multiple **repos**.

A repo contains related code and other files that are **changed**, **branched**, and **released** together, along with the full history of all files.

Repos are managed in a git server; you create a local copy to work in using `git clone`

A repo contains **commits**, **tags**,and **heads** (aka branches)

A repo has a pointer to a **remote**

Einstein repos are managed on a BitBucket server at https://einstein-bitbucket.medtronic.com

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Commits

A commit is a version of the files in the repo, or a **snapshot** of the state of the repo, including directories, files and symlinks.

Each commit contains a full set of files & directories.

- Git does not store history as deltas; it stores complete snapshots. When you ask for history and it shows the history as deltas, those deltas are generated on-the-fly.

Commits are immutable. They are named by a checksum of the commit contents – change any part of a commit, and you have a new, different commit.

- The checksum algorithm is **SHA1**, and the checksum can be called "the SHA1 of the commit". A SHA1 checksum is a 40-hex-digit string that looks something like this: 723f670089fef8fbd7a925f71c2bab9662c4da7e

Commits can be signed; we don't sign commits.

# Symlinks in repos

Symlinks are supported in git; however:

- Windows clients do not typically support symlinks because the underlying Windows filesystems do not support symlinks.
- In a multi-platform environment like Einstein, symlinks are of limited value because they don't work on Windows. You should avoid adding symlinks to an Einstein repo.

# Commit contents

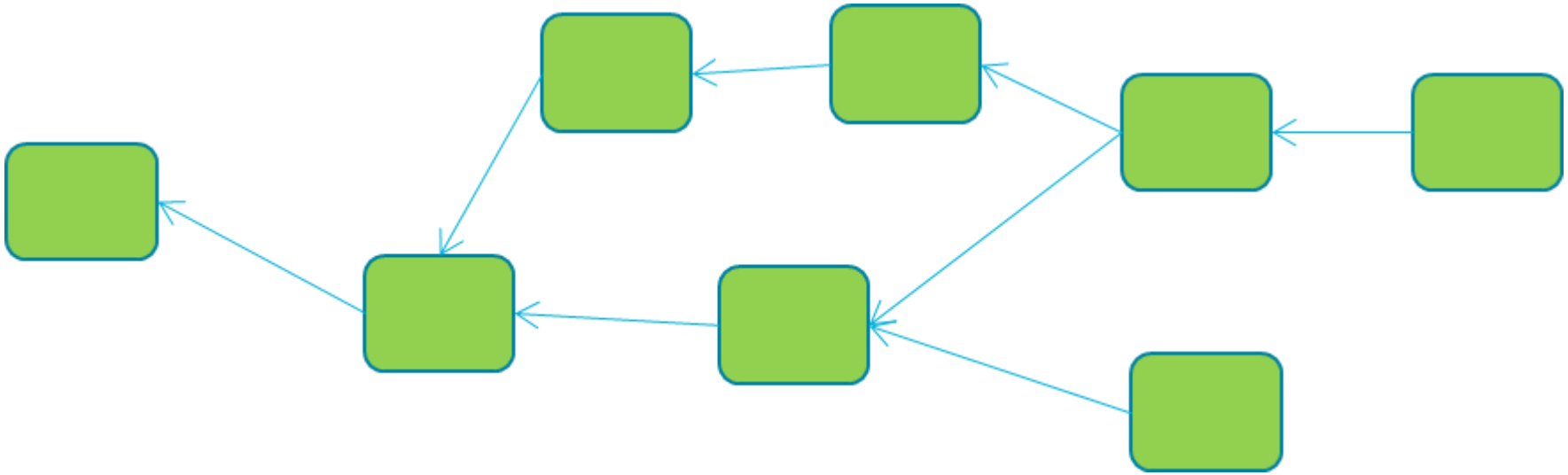
A commit contains:

- Directories, files and symlinks
- Parent commit SHA1s if any
- Author with timestamp
  - Author is the original author of the changes; author should be preserved if the change is applied as a patch
- Committer with timestamp
  - Committer is the person that commits the change; committer will change if the change is applied as a patch
- comment
- optional signing information

# Commits and history

Each commit has one or more predecessor pointers.

Overall, commits form a **Directed Acyclic Graph (DAG)** with one source and many sinks



# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Remotes

A **remote** is a pointer to a remote repo.

When you create a local repo using **git clone**, it automatically creates a remote named **origin**.

Operations like **git pull** and **git push** use the remote to know where to pull from or push to.

To list the remotes: `git remote`

To list the remotes with their URLs: `git remote -v`

# Tracking branches

Tracking branches reflect what the local repo thinks the branches look like in the remote repo.

Tracking branches are not updated manually - they are only updated automatically by git pull, git fetch, and git push.

Tracking branches have names like **origin/develop** or **origin/feature/TAP-1234**

To list tracking branches:

- `git branch -r`

# Interacting with remotes

`git clone`

- Creates a local clone and creates the remote “origin”
- Creates tracking branches for all remote branches
- Creates a local branch for the default branch in the remote, and ties it to the remote branch.

`git pull`

- Pulls new commits from the remote into tracking branches.
- For the current (local) branch, merges any new commits from the remote into the local branch

`git fetch`

- Just pulls new commits into tracking branches.

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

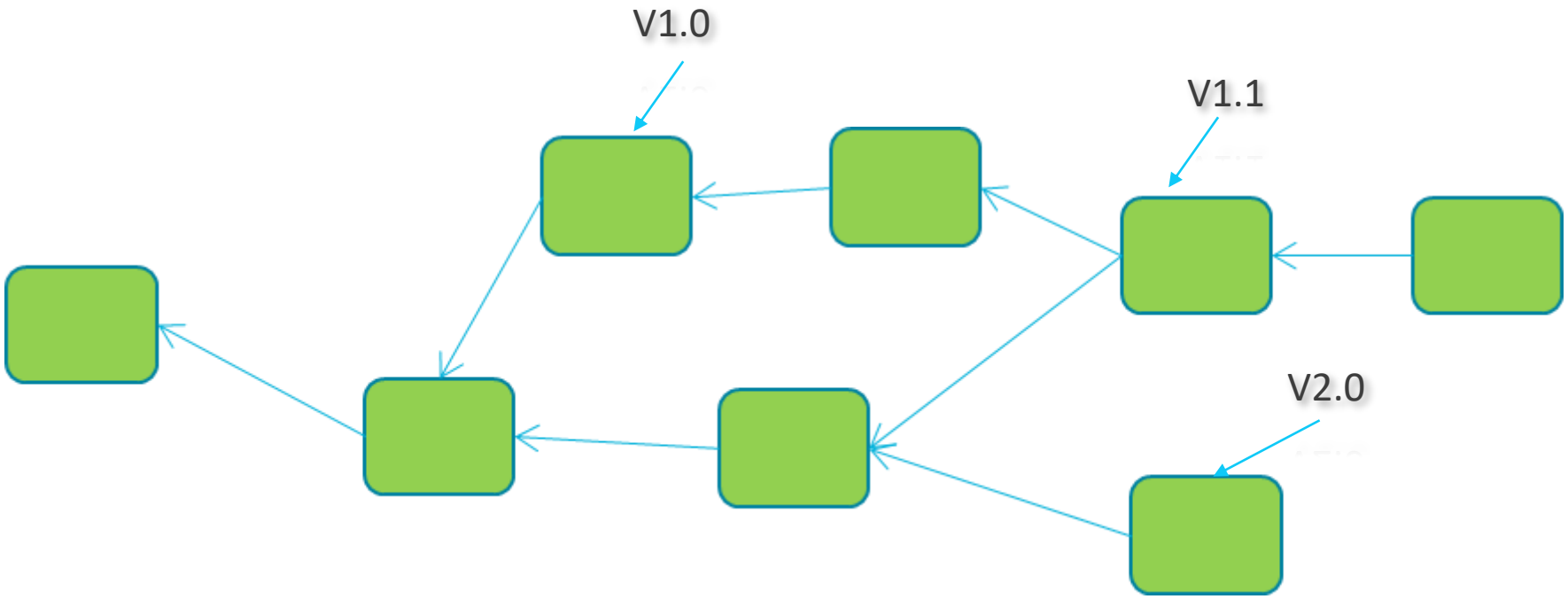
Review

Medtronic

# Tags

A **tag** is a label for a commit.  It typically doesn't change once it is created.

To list tags: `git tag`




Medtronic

# Two types of tag

There are two types of tag:

- **Lightweight**. This is a simple label for a single commit. It has a name, but nothing else.
- **Annotated**. An annotated tag is more complex and contains more:
  - The name of the tag
  - The tagger's name
  - A timestamp
  - A comment
  - A commit
  - Optional signing information. We don't sign annotated tags.

Annotated tags have two SHA1s (just to be confusing): the SHA1 of the commit that it references, and the SHA1 of the annotated tag object.

# Tags and remotes

When you first clone a repo, all the tags in the remote are copied to your local repo.

When you pull or fetch, any new tags in the remote are copied to your local repo.

Tags are **not** immutable!

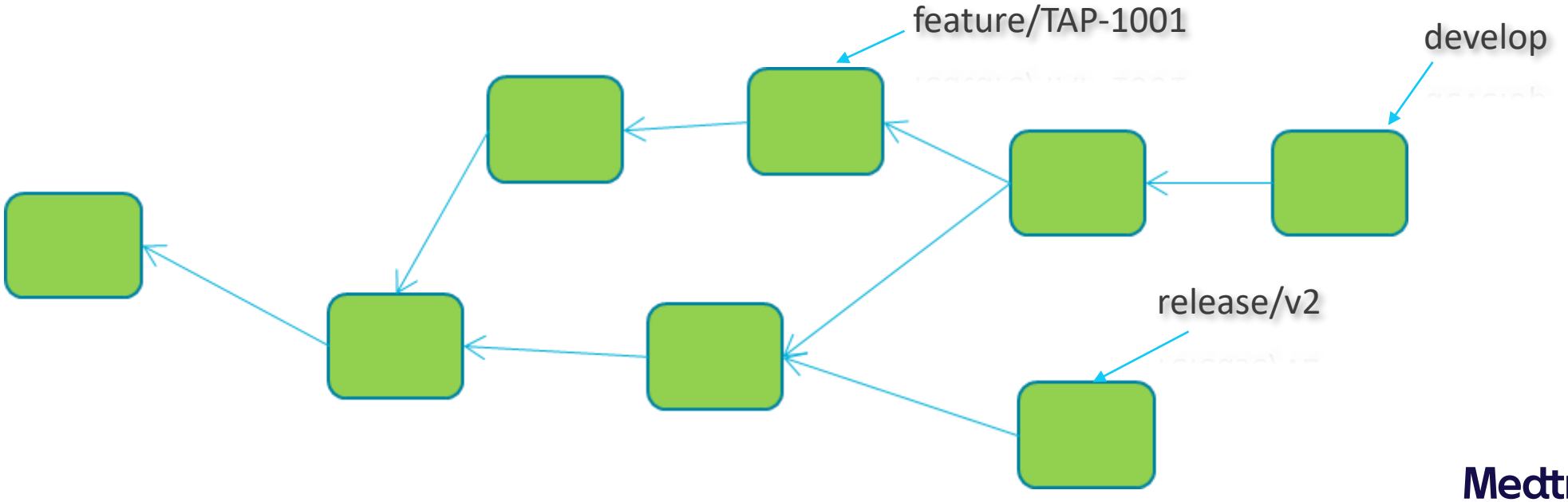
- They **can** be changed in the remote repo or in your local repo
- **Changed tags are not fetched!** If a tag is changed anywhere then that change is **not** propagated to existing repos with fetch or pull or push.
- Even though they can be changed, we don't change them. We pretend they are immutable.

# Heads

Git does not have branches!

Git has heads, which are very similar to lightweight tags.

In common with git documentation, I will use the terms **head** and **branch** interchangeably - sometimes it makes sense to talk about heads, sometimes the term branches is more natural.




Medtronic

# Heads

A repo has a current HEAD and possibly other heads.

To list the heads in the current repo: `git branch` (yeah, I know...)

- The current head is marked with a * in the output:

```
$ git branch
    develop
  * feature/TAP-111668
```

You can also list the heads in the remote: `git branch -r`

# Local and remote branches

Local branches are different from remote branches, but they can be linked.

When you clone a repo, it automatically creates a local branch with the same name as the **default** branch in the remote repo, and links the new local branch to the remote that you cloned from.

- It also automatically creates a tracking branch for each remote branch.

To create a new branch and link it to a remote:

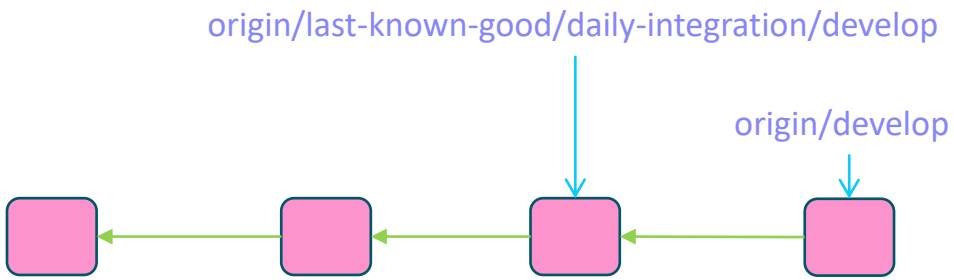
- First create a new local branch: `git branch feature/TAP-101`
- Make some changes on the new local branch and create a commit: `git commit` …
- Then when you push, tell git which remote to push to, and tie the local branch to that remote: `git push --set-upstream origin feature/TAP-101`

# origin/last-known-good/daily-integration/develop

origin/last-known-good/daily-integration/develop (**OLDD**) is a remote head, that trails origin/develop.

Think of it as a tag that changes - we manage it as a head so that git fetch and git pull will pull changes (remember - git fetch does not pull tag changes).

**All work on local branches should be based on OLDD, not origin/develop**.



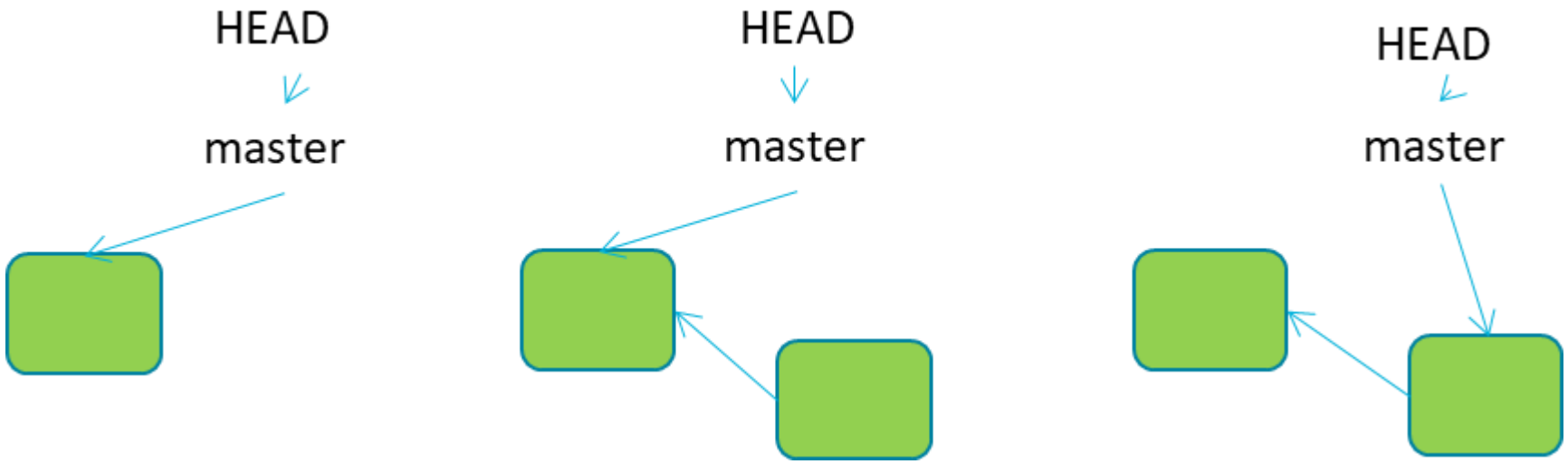

Medtronic

# Adding a commit to a branch

Branches are like lightweight tags, except they update when you add a new commit.

Each repo has a special meta-label **HEAD** that names the current local branch.

When you create a new commit, when the commit is created then the branch named by HEAD is updated to point to the new commit




Medtronic

# Default branch names

Historically, the default branch name in a new git repo was **master**

The computing world recently went through an awakening, and terms like "master" and "slave" are being dropped in favor of terms like "main" and "agent"

- For example, bamboo and jenkins don't have "build slaves" any more – they have "build agents".

The new default branch name is **main**.

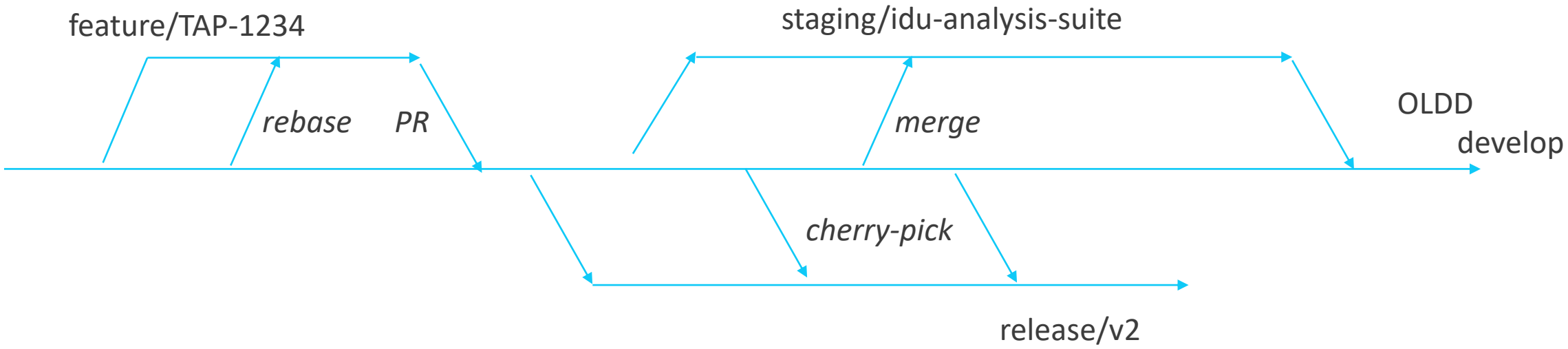
In Einstein the default branch name has been changed to **develop**. Depending on the specific repo and its history, you may see that the default branch name is any one of the three defaults.

# Branch types

## In Einstein we have several branch types:

- The **development branch**, named **develop**, which contains the latest version of code for the next release.
- **origin/last-known-good/daily-integration/develop** is a pseudo-branch that trails develop, and that contains the latest “known-good” version of code.
- **Feature branches**, with names like **feature/TAP-12345** for active development of a new feature or bugfix.
- **Staging branches**, with names like **staging/plug_and_play**. These are for long-lived project based development that is not yet approved for a mainline.
- **Release branches**, with names like **release/v2**, that are used for stabilization and release of a release.

# Branch types




Medtronic

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Workspaces

A **workspace** is the term for a local clone of a repo.

You create a workspace by cloning a bitbucket repository:

- `git clone ssh://stash:7999/ein-prod/ein_ng`

Initially the workspace has the same branch as the default branch of the remote repo, i.e. develop.

# Optimizing clone

Cloning is expensive because it downloads the entire history of the repo

Git has a couple of mechanisms to help with this:

- Create multiple **worktrees** for a single repo clone.
- Use a **reference repository** .

# Worktrees

A **worktree** is a separate working tree containing the repo files.

Worktrees share the same .git directory, so after the initial clone, creating a new worktree is fast - about **3 seconds** for ein_ng.

To create a new worktree in directory ../TAP-1234:

- `git worktree add ../TAP-1234 origin/last-known-good/daily-integration/develop`

To list worktrees:

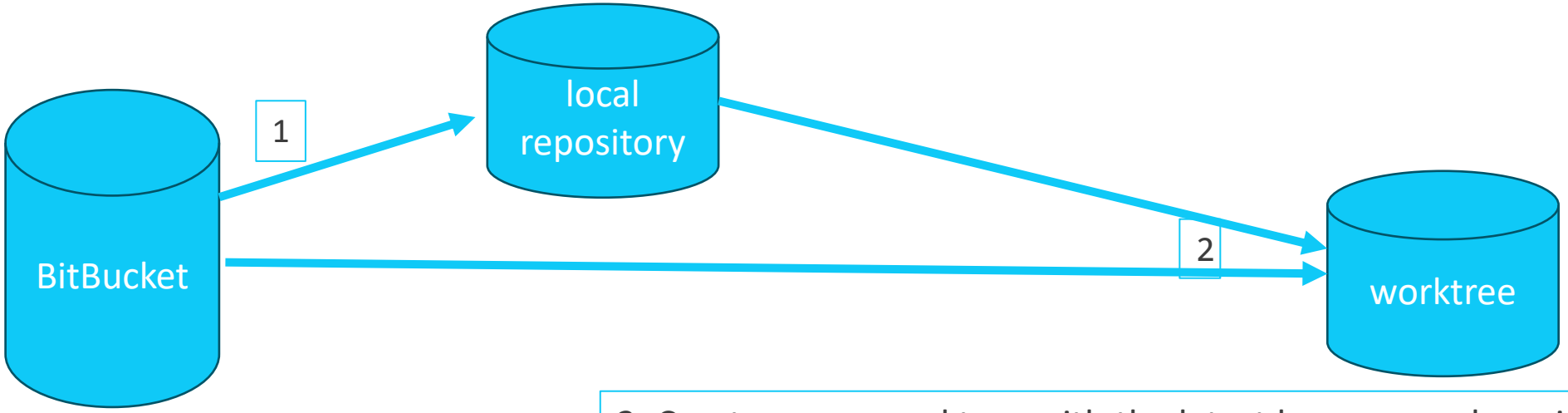
- `git worktree list`

To remove the current worktree:

- `git worktree remove .`

# Worktrees

1. Create a local repository:
```
cd ~/.cache/git; git clone ssh://stash:7999/ein-prod/ein_ng
```




2. Create a new worktree with the latest known-good version, ready for branch creation
```
git worktree add ~/TAP-1234 origin/last-known-good/daily-integration/develop
```

Medtronic

# Reference repository

**A reference repository** is a full clone of the remote repo, including all files and directories.

- Expensive to clone initially, but this is a one-time cost.

When a local clone is created using a reference repository, any objects that exist in the reference repository are fetched from there, **not** the remote repository

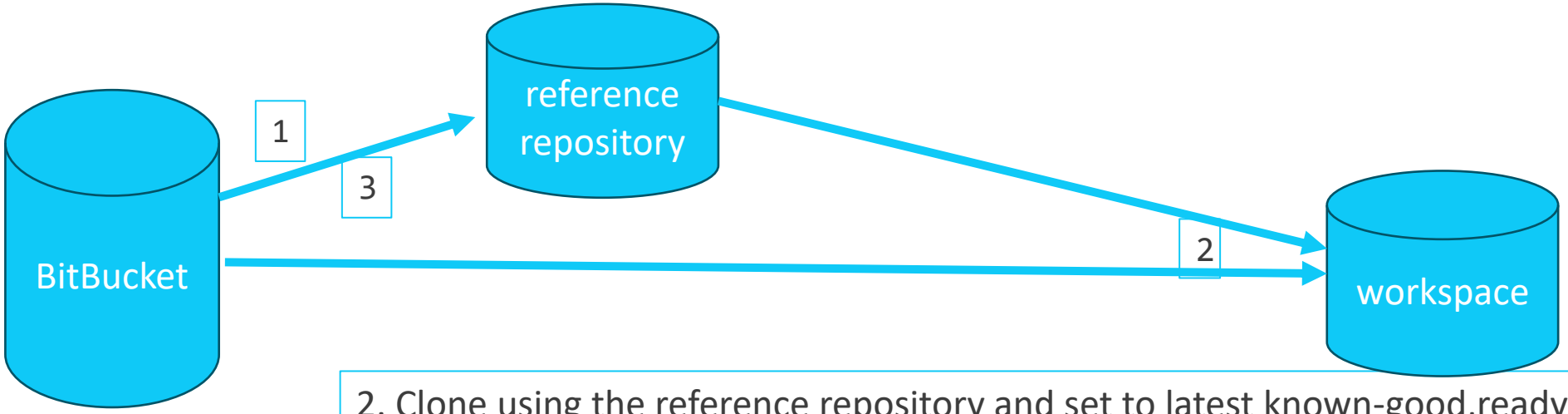
Keep the reference repository up-to-date by running `git fetch` on a regular basis

Time to clone ein_ng over VPN using a reference repository is about **5 seconds**.

# Reference repository

1. Create a local reference repository:

```
cd ~/.cache/git; git clone --mirror ssh://stash:7999/ein-prod/ein_ng
```




2. Clone using the reference repository and set to latest known-good,ready for branch creation

```
git clone --reference ~/.cache/git/ein_ng.git ssh://stash:7999/ein-prod/ein_ng
cd ein_ng; git checkout origin/last-known-good/daily-integration/develop
```

3. Keep the reference repository up-to-date

```
cd ~/.cache/git/ein_ng.git; git fetch
```

Medtronic

# Workspace strategies

**One workspace, many local branches**

- Work on multiple features/bugfixes in one workspace, using local branches to separate the changes
- Advantages: only need to clone once
- Disadvantages: can be confusing; difficult to work on multiple activities simultaneously

**One workspace, many worktrees**

- Use worktrees to separate activities
- Advantages: only need to clone once
- Disadvantages: more complex git environment; multiple inter-related repos and trees

**One workspace per ticket**

- For each ticket, create a new local workspace
- Advantages: can work on multiple activities simultaneously; easy to clean up once an activity is completed
- Disadvantages: need to clone for each activity,  but this is faster with a reference repository.

**Pick a strategy that you understand and feel comfortable with**

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

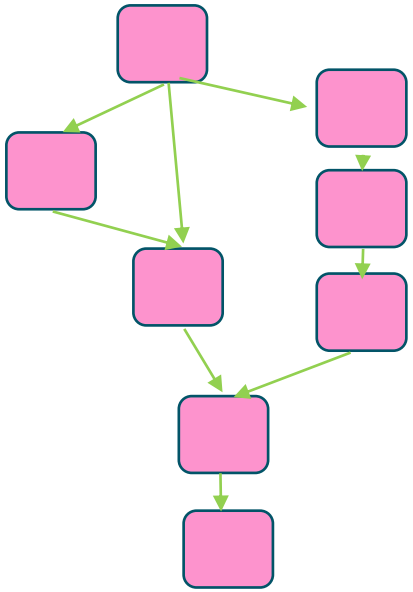
# Merging and squash

Since git commits form a DAG, history can quickly become complicated.

Rather than have a complex history with a lot of branching and merging in the DAG, it is better – if possible – to have a single-line history. Merging and squash are tools to help us keep history simple.

Good:

Bad:



# Merging

There are three types of merge between branches:

- Fast forward merge
- Normal merge
- Rebase merge

Squash is a technique that uses merge commands to collapse history on a single branch

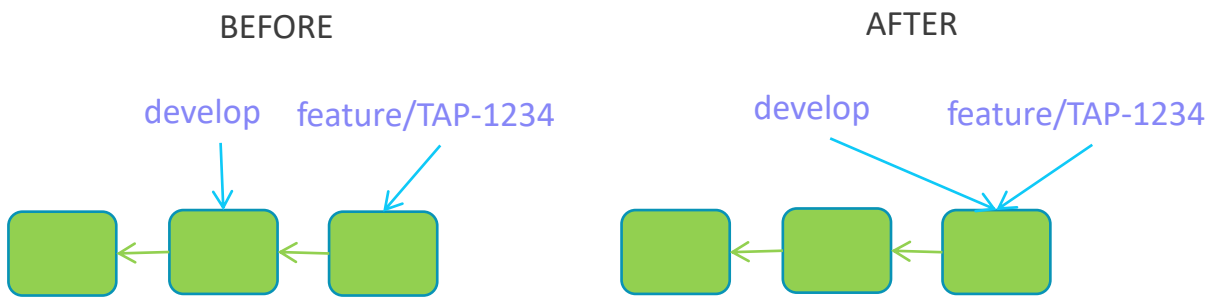
# Fast forward merge

Simplest “merge”, when from-branch is a strict successor of to-branch.

Situation automatically detected.

- `git checkout develop`
- `git merge feature/TAP-1234`

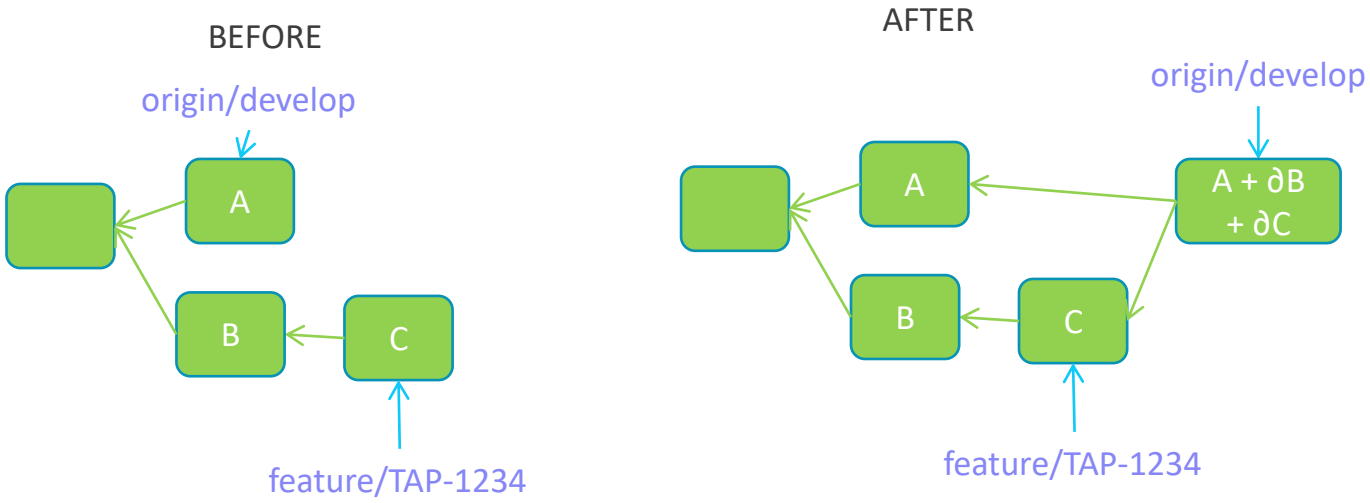
If you were actually merging to develop, this would be done with a PR.



# Normal merge

Commits on from-branch are converted to patches, that are applied to to-branch and are packaged into a new merge commit with multiple predecessors.

- `git checkout feature/TAP-1234`
- `git merge origin/develop`

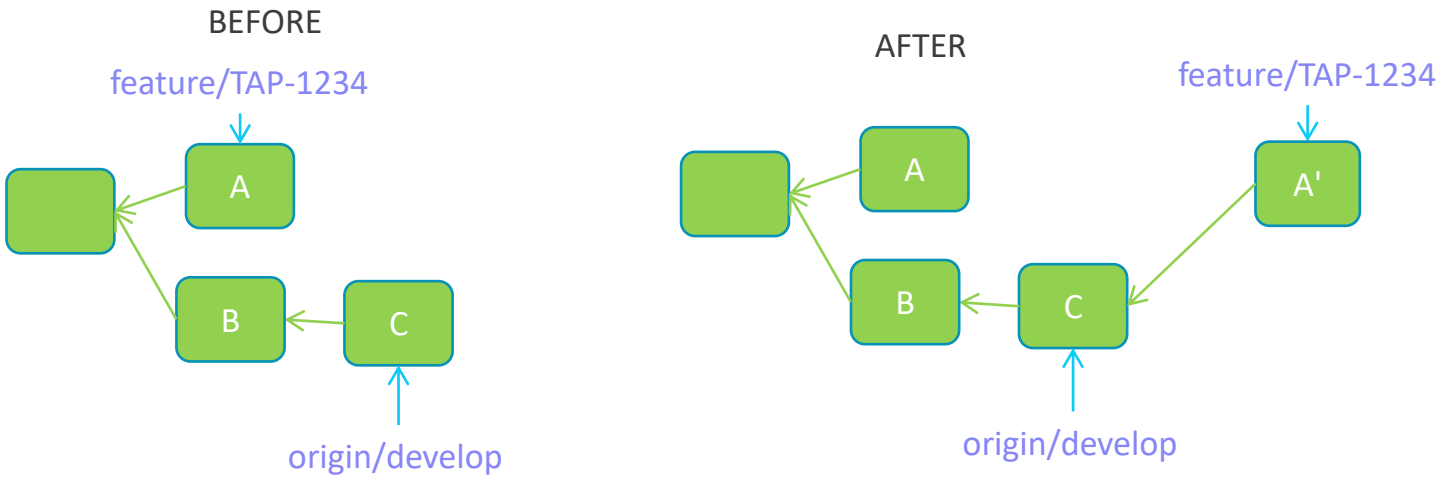



Medtronic

# Rebase

Commits on this branch are converted to patches.  Patches are replayed on remote branch; each patch (by default) creates a new commit.

- `git checkout feature/TAP-1234`
- `git rebase origin/develop`

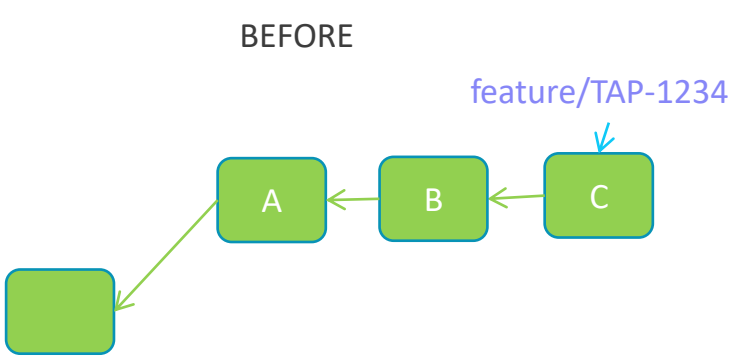


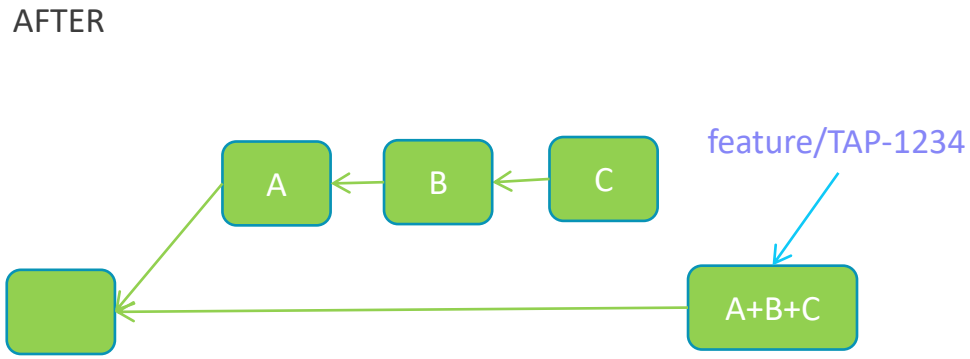

Medtronic

# Squash

Commits on this branch are squashed into a single commit.

- `git checkout feature/TAP-1234`
- `git rebase –i HEAD^3`




Medtronic

# Squash - how do I know which commit to squash to?

Method 1: `git log`

Your branch commits

Branchpoint

```
commit 9baa9059484985471e435e985a247a1ad1f1f24c (HEAD -> feature/TAP-111668, origin/feature/TAP-111668)
Author: Lezz Giles <lezz.p.giles@medtronic.com>
Date:   Thu Apr 4 15:55:55 2024 -0400

    TAP-111668: Use pagination when getting agent assignment details

commit 281f3251e0063ec9be2cd8d1c41becf841a585fc
Author: Lezz Giles <lezz.p.giles@medtronic.com>
Date:   Thu Apr 4 13:25:26 2024 -0400

    TAP-111668: more changes from PR review

commit 30b941ea0b86c5ffc4e17d16143a29a56dcdf85e
Author: Lezz Giles <lezz.p.giles@medtronic.com>
Date:   Thu Apr 4 13:18:34 2024 -0400

    TAP-111668: install local copy of packer package

commit 960e2b117bf73e65027608843dc84b621ec13345
Author: Lezz Giles <lezz.p.giles@medtronic.com>
Date:   Fri Mar 15 15:06:26 2024 +0000

    TAP-111668: terraform to manage build agent VMs

commit a721b216c207fda63e3517fecfdf3947bf8ddf09 (origin/develop, origin/HEAD)
Author: Varun Pandey <varun.pandey@medtronic.com>
Date:   Mon Mar 25 12:52:39 2024 -0400

    Merge pull request #26 in TOOLS/devops-infrastructure from feature/TAP-95211-report-and-graph-bamboo-bas

    Squashed commit of the following:

    commit 0bfb655dc763dbbba46bd70c37cd0f7666ea6917
    Author: THCG\pandev5 <varun.pandey@medtronic.com>
    Date:   Mon Mar 25 18:20:53 2024 +0530

        TAP-95211 : Updated for Prometheus.
```

`git rebase -i origin/develop`

or

`git rebase -I a721b21`

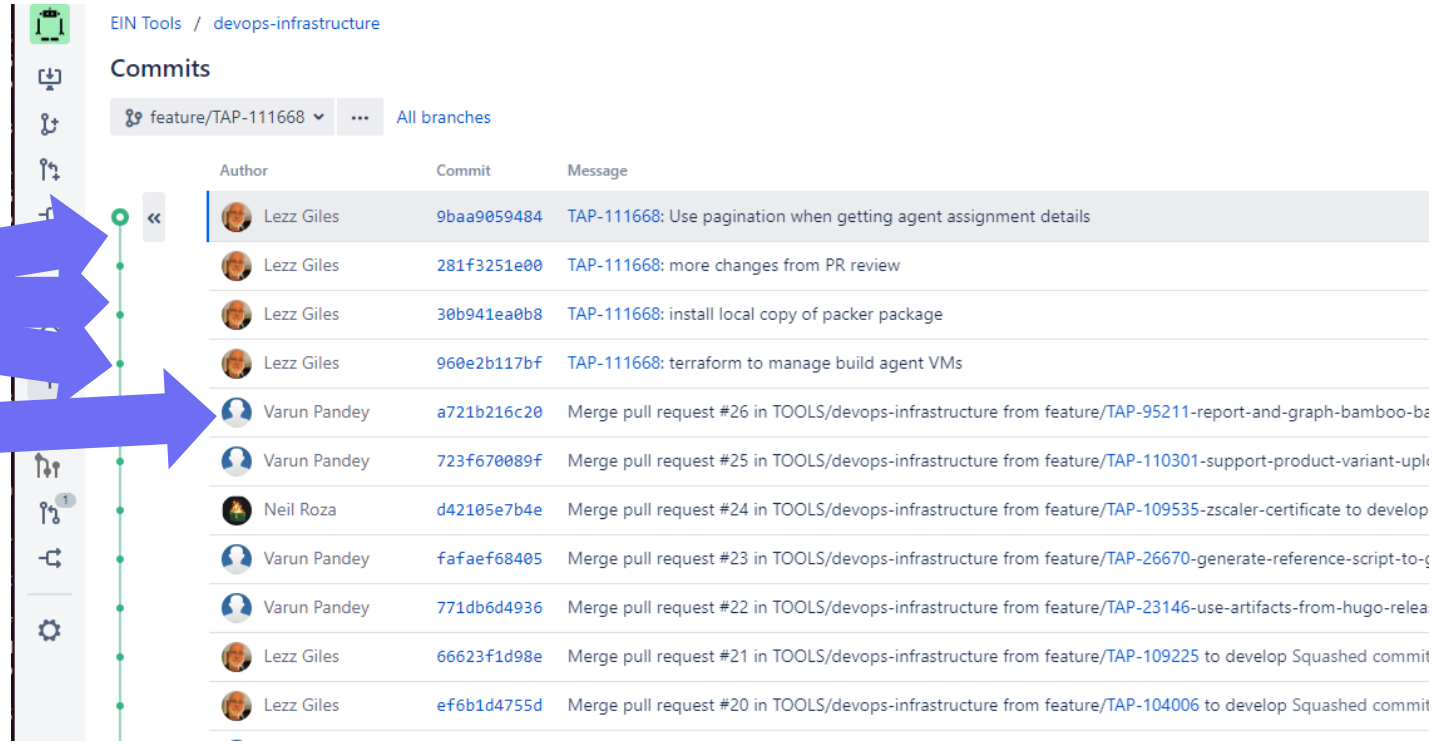
Medtronic

# Squash - how do I know which commit to squash to?

Method 2: Bitbucket

Your branch commits

Branchpoint




```
git rebase -I a721b21
```

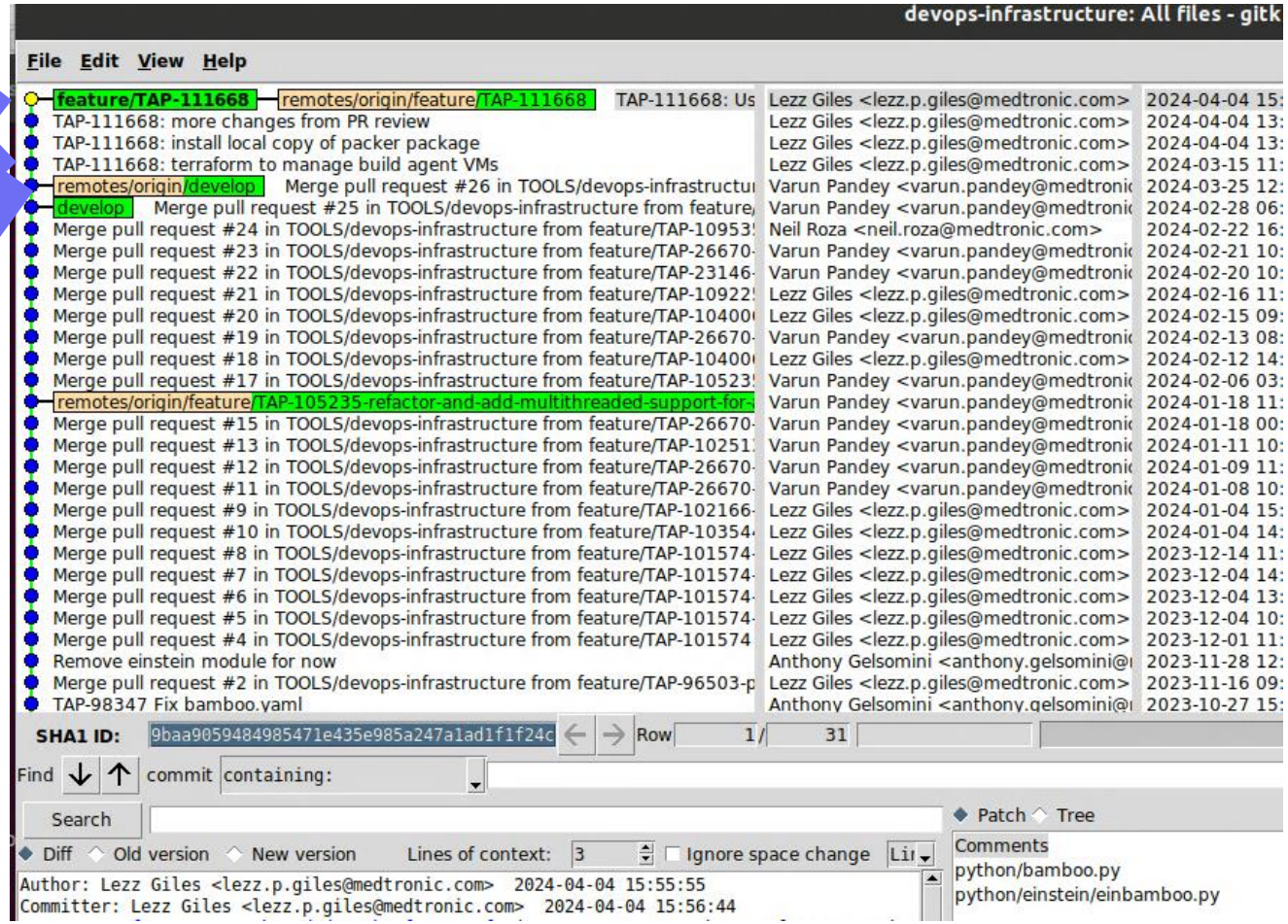
Medtronic

# Squash - how do I know which commit to squash to?

Method 3: `gitk`

Your branch commits

Branchpoint




```
git rebase -i origin/develop
```

or

```
git rebase -I a721b21
```

Medtronic

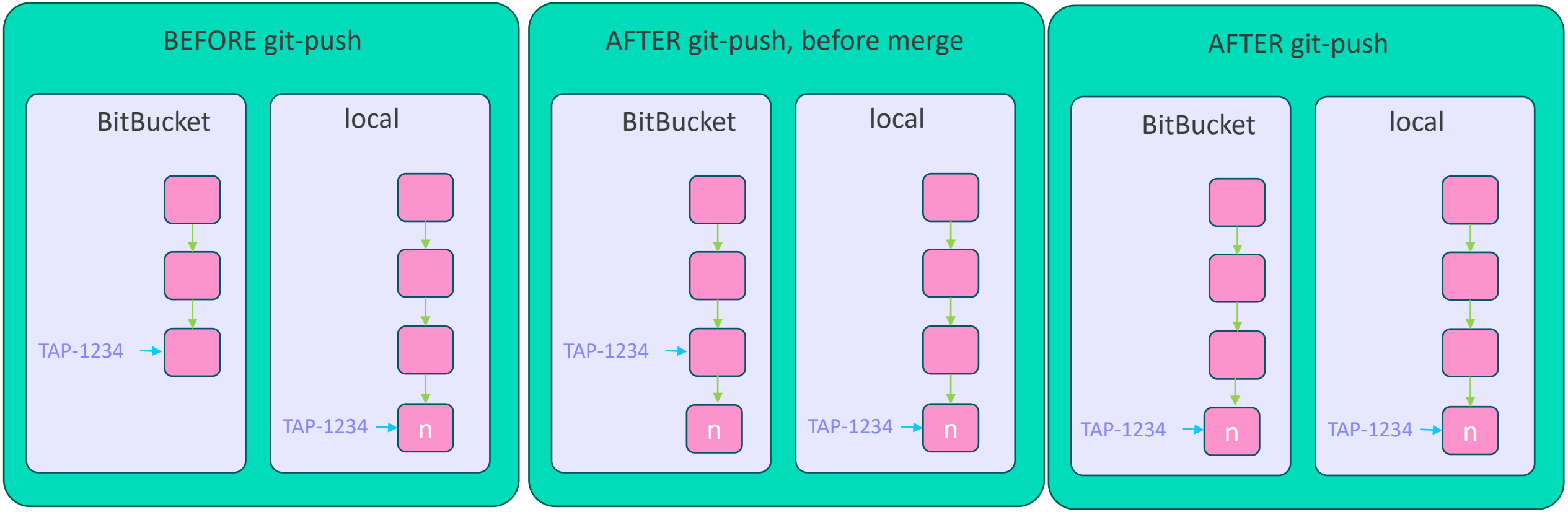
# Squash - how do I know which commit to squash to?

Method 3: `git merge-base:`

```
git rebase -I `git merge-base HEAD origin/develop`
```

# git push

When you use `git push` to push changes to BitBucket, it expects a **fast-forward** merge:



# git push with squashed commits

If you have rebased your branch, either to pick up changes from develop or to squash changes, your branch has diverged from the remote branch.

- `git push` will fail if the merge is not a fast-forward
- Need to use `git push -f` to force the update of the remote branch when the branches have diverged.

Medtronic

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Pull Requests

A Pull Request is the mechanism in BitBucket for delivering changes to develop and other branches.

- First, the code to be delivered is developed and tested on a feature branch.
  - The changes should be built on origin/last-known-good/daily-integration/develop (OLDD)
- The PR is created, which automatically creates a code review.
- Reviewers create comments which might require more changes.
- The changes are implemented on the feature branch and are automatically added to the PR.
- When all changes have been implemented, the feature branch should be rebased to develop instead of OLDD
- When reviewers approve the PR and Bamboo builds on the feature branch pass, the PR is merged.

The step-by-step procedure is covered elsewhere.

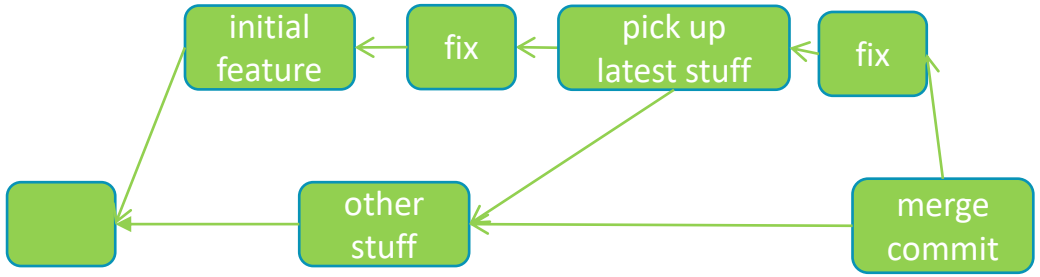
# Feature branches and merges and KISS (1)

Most commits are made on feature branches, and are then delivered to the development branch with a PR.

We do not want to mess up the development branch with hundreds of intermediate commits; better to end up with **one commit per feature**.

How do we do this?

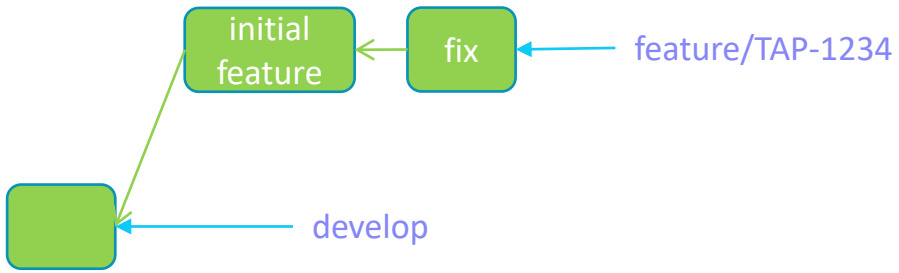
BAD:




GOOD:

# Feature branches and merges and KISS (2)

First, create a feature branch and develop the feature, possibly with multiple commits

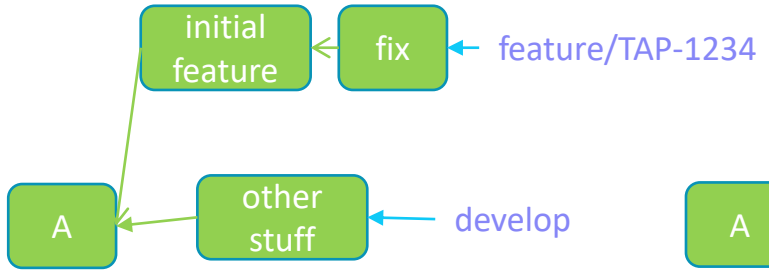


# Feature branches and merges and KISS (3)

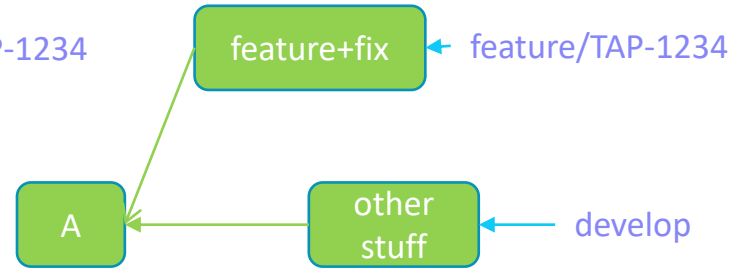
There is some other work added to **develop**, so squash and rebase:

- `git rebase -i A`
- `git rebase origin/develop`

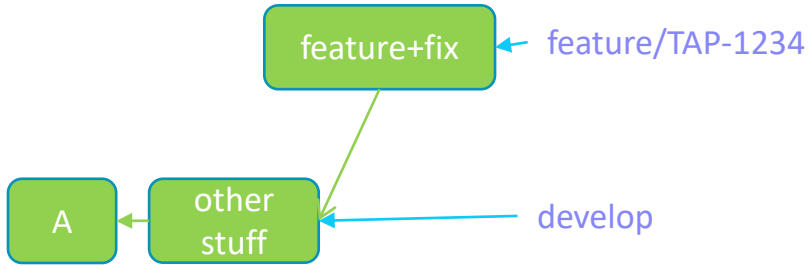
BEFORE:




AFTER SQUASH:




AFTER REBASE:

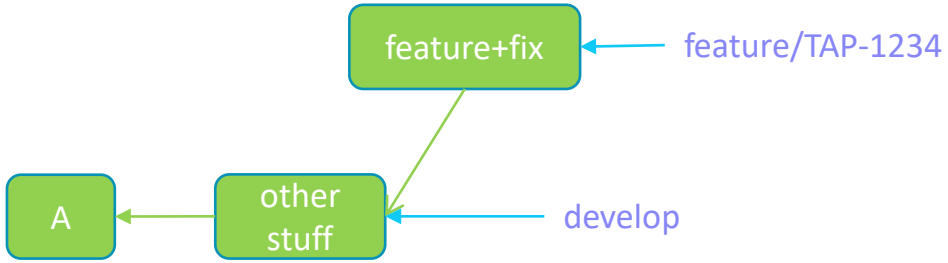



If you squash before rebasing then you only need to resolve merge conflicts in the rebase once; if you rebase with multiple commits on your branch then you may have to resolve merge conflicts multiple times.

# Feature branches and merges and KISS (4)

The PR (pull request) becomes a fast-forward merge!

BEFORE PR:




AFTER PR:

Medtronic

# Do not merge to feature branches

It is valid to merge out from develop to a feature branch instead of rebasing, to pick up more recent changes.

However if you do merge to a feature branch, don’t use rebase later (and if you rebase, don’t merge later)

- Mixing merge and rebase will lead to merge conflicts and will complicate the overall process and the commits DAG.

Moving forward, always just rebase, do not merge.  Merge causes complex history, which we want to avoid.

# Overview

Concepts:

- Repos
- Commits
- Remotes
- Tags and heads

Feature branch tasks:

- Workspaces and cloning
- Merging and squash
- PRs

Review

Medtronic

# Review

The Einstein codebase is stored in git repos

Changes are made with commits on feature branches.

Commits form a DAG, with tags and heads/branches to help manage the DAG.

There are several strategies for avoiding the cost of cloning large repos; including using reference repos and using one local repo for multiple activities.

Fast-forward merges and squash are techniques for simplifying the DAG.

Feature branches are delivered to integration branches using PRs.

Step-by-step procedures are in the Confluence **Einstein Software DevOps** folder

Also in the Confluence Einstein Software DevOps folder:

- This Powerpoint presentation
- Einstein Git quick reference card

# Thank you

Medtronic